Лабораторная №3. План подготовки

Тема: модель данных OpenMP. Главная задача на защите - для каждой переменной определить, какой объект она обозначает на каждой нити, кто его инициализирует, кто имеет право менять значение и что останется после завершения области.

Короткое описание своей работы

«Я показал private, shared, firstprivate, threadprivate и copyin по примерам 12-16. Отдельно добавил lastprivate, reduction и copyprivate, чтобы продемонстрировать возврат результата и передачу между нитями. В private не читаю неинициализированный int; в shared ограничиваю команду, чтобы индекс не вышел за массив. Ответы на шесть вопросов вынесены в документ, а не в вывод программы».

Механизм	Смысл в одной фразе
shared	Один общий объект; синхронизацию нужно обеспечить отдельно
private	Отдельный экземпляр без копирования внешнего начального значения
firstprivate	Отдельный экземпляр с инициализацией внешним значением
lastprivate	Возврат значения последней логической итерации
reduction	Приватные аккумуляторы и объединение заданной операцией
threadprivate	Экземпляр переменной на нить с особым временем жизни
copyin	Начальная рассылка экземпляра мастера для threadprivate
copyprivate	Рассылка результата single в приватные экземпляры участников

Как демонстрировать

Главное меню **3**, затем **1-8** по одному примеру или **9** для всех. Для каждой демонстрации заранее сказать, какими будут значения до области, внутри неё и после. Порядок строк не фиксирован. Текущий код использует четыре запрашиваемые нити; объяснять результаты нужно с учётом фактически созданной команды.

Основа: предоставленная лабораторная №3, примеры 12-16 и шесть вопросов; текущий Lab3.cpp. Материал рассчитан на C++17 и MSVC /openmp. Более новые возможности OpenMP не следует автоматически считать доступными в этом режиме компиляции.

1. Правила модели данных

Имя переменной не равно одному объекту

Если value объявлена shared, все участники обращаются к одному объекту. Если value указана в private, одно и то же имя в блоке соответствует отдельным объектам разных нитей. Поэтому фраза «все нити пишут value» недостаточна для вывода о гонке: сначала нужно понять, пишут ли они в одну ячейку памяти.

Где объявлено	Обычное правило для примеров этой работы
Автоматическая переменная снаружи parallel	Обычно shared, если не задано другое допустимое правило
Автоматическая переменная внутри блока parallel	Отдельный объект при выполнении блока каждой нитью
Счётчик распределяемого omp for	Приватный по предопределённому правилу
Глобальная или static-переменная	Обычно shared; место объявления внутри блока не делает static приватной
threadprivate-переменная	Отдельный экземпляр для нити согласно директиве
Память, выделенная динамически	Приватность указателя не создаёт приватную копию памяти по адресу

Это ориентиры, а не полный стандарт для всех конструкций. default(shared) задаёт обычное поведение для соответствующих переменных; default(none) требует явно описать атрибуты переменных, у которых они не предопределены. В используемой версии C/C++ не нужно переносить варианты default из примеров на других языках без проверки поддержки.

shared не означает «безопасно»

Гонка возникает при конфликтующих несинхронизированных обращениях к одной памяти, когда хотя бы одно обращение - запись. Только чтение неизменяемого значения многими нитями допустимо. Записи в разные элементы обычного int-массива не являются записью в один объект-элемент. Даже запись одинаковых значений в одну ячейку требует корректного доступа.

Барьер разделяет фазы работы и обеспечивает нужные эффекты видимости памяти, но не собирает приватные копии и не исправляет уже случившуюся гонку. volatile в C++ не заменяет atomic, critical, reduction или корректные барьеры.

2. private: пример 12

Точный разбор showPrivate()

До parallel создаётся обычная value = 1. В директиве private(value) запрашиваются приватные экземпляры. У каждого обычного int нет начального значения из внешнего объекта. Поэтому первым действием нить присваивает value = omp_get_thread_num(), и только после этого выводит значение в critical.

```
int value = 1;
#pragma omp parallel num_threads(4) private(value)
{
    value = omp_get_thread_num();
    // Чтение теперь корректно: значение уже присвоено.
}
// Внешняя value по-прежнему равна 1.
```

Место	Значение
Внешний объект до области	1
Приватные объекты до присваивания	Читать обычный int нельзя: инициализации нет
Нити 0, 1, 2, 3 после присваивания	0, 1, 2, 3 соответственно
Внешний объект после области	1

Почему не печатается «мусор на входе»

В методичке напечатана неинициализированная private-переменная. В используемом C++17 чтение такого int приводит к неопределённому поведению. Нельзя описывать результат как корректно полученное случайное число и делать выводы по одному запуску. Рабочий пример исправлен: неизменность внешней value показывает приватность без ошибки языка.

Даже нить 0 внутри области работает с приватным экземпляром value, а не получает право менять внешний объект. Это отличает private/firstprivate от случая threadprivate, где последовательная часть и начальная нить используют её постоянный экземпляр.

Что изменится, если заменить опцию

С firstprivate каждая нить начала бы с 1, а внешняя value всё равно осталась бы 1. С shared записи value = номер нити конфликтовали бы: один общий объект изменяли бы несколько участников. Даже если вывод защищён critical, сама запись вне critical не становится защищённой автоматически.

private для объекта класса не означает «битовая копия без конструктора»: действуют правила C++ и требования OpenMP к типу. На защите этого примера достаточно точно разобрать обычный int, а не переносить его неинициализированное состояние на любой класс.

3. shared: пример 13

Как меняется массив

showShared() создаёт int values[10] = {}, то есть десять нулей. Команда запрашивает четыре нити. Нить k записывает values[k] = 1. После parallel выводятся значения и проверяется, что первые actual элементов стали единицами, а остальные остались нулями.

```
int values[10] = {};
int actual = 0;
#pragma omp parallel num_threads(4) shared(values, actual)
{
    values[omp_get_thread_num()] = 1;
#pragma omp single
    actual = omp_get_num_threads();
}
```

При фактической команде из четырёх нитей результат: **1 1 1 1 0 0 0 0 0 0**. Общий массив один; это не объединение четырёх приватных массивов. При выходе из parallel все записи участников завершены, поэтому последующий последовательный обход читает готовый результат.

Почему здесь нет гонки и зачем single

У каждой нити уникальный номер в команде, следовательно, каждая пишет в отдельный элемент int. Синхронизация каждой такой записи не требуется. actual, напротив, одна общая переменная, и single оставляет ей одного писателя. Записать туда одно и то же значение одновременно всем нитям - не правильная альтернатива single.

В исходном примере массив имел размер 10, а команда не ограничивалась. На машине с 12 участниками обращение values[10] или values[11] вышло бы за границы. num_threads(4) делает индекс безопасным в текущем примере, где actual не больше запроса. В универсальном алгоритме размер данных и размер команды нужно согласовывать явно.

Если спросят про false sharing

Разные элементы могут попасть в одну кэш-линию. Тогда процессоры передают владение линией друг другу, и запись может работать медленнее. Это false sharing: проблема производительности, а не гонка обращений к одному int. Не нужно добавлять critical для исправления результата, который и так корректен; оптимизация размещения данных - отдельная задача.

Если всем нитям вместо values[thread] дать values[0], возникнет конфликт записей. Если заменить запись простым чтением неизменяемого values[0], конфликт сам по себе исчезнет.

4. firstprivate: пример 14

Как работает showFirstprivate()

Внешняя value равна 1. firstprivate(value) создаёт отдельные экземпляры и инициализирует их внешним значением. Поэтому сохранённая before равна 1 у каждого участника. Потом нить присваивает value свой номер. После parallel внешняя value по-прежнему равна 1: обратного копирования эта опция не делает.

Нить	before на входе	value после присваивания	Внешняя value после области
0	1	0	Один общий внешний объект: 1
1	1	1	Он не заменяется значениями копий
2	1	2	
3	1	3	

В программе ошибки входного значения подсчитываются в errors с reduction(+:errors). Каждый участник увеличивает только свой аккумулятор, затем они суммируются. Это служебная проверка; firstprivate не нуждается в reduction, чтобы передать саму value.

Указатель: копируется адрес, не память

```
int common = 0;
int* pointer = &common;
#pragma omp parallel num_threads(4) firstprivate(pointer)
{
    // pointer у каждой нити свой.
    // *pointer указывает на один общий common.
}
```

Скопировать значение указателя означает получить несколько экземпляров одного адреса. Данные по этому адресу не становятся независимыми. Несинхронизированная запись в *pointer из всех нитей всё ещё могла бы стать гонкой. Для независимых данных нужны отдельные целевые объекты или непересекающиеся диапазоны.

Вопрос с подвохом

«Если начальная value = 1, то почему private не получает 1?» Потому что private задаёт только приватность, а firstprivate добавляет копирование начального значения. «А если нужна и начальная копия, и возврат результата цикла?» Для допустимой переменной распределяемого цикла могут сочетаться firstprivate и lastprivate в соответствии с правилами конструкции; это не превращает объект в shared.

Передача firstprivate обычного класса подчиняется требованиям к копированию объекта. Пример со std::string в рекурсии лабораторной №2 показывает такую инициализацию параметра, но не означает автоматическое глубокое копирование всего, что класс может хранить по указателям.

5. threadprivate: пример 15

Отдельный экземпляр на нить

threadValue находится в анонимном пространстве имён и имеет статическую длительность хранения. Директива threadprivate делает экземпляр отдельным для каждой нити. В отличие от private на одном parallel, это механизм состояния нити, которое при условиях разрешено использовать и в следующей области.

```
namespace
{
    int threadValue = 0;
#pragma omp threadprivate(threadValue)
}
```

Подготовительная команда обнуляет свои экземпляры. Это важно при повторном выборе пункта меню: рабочие потоки могут уже иметь значения от copyin или предыдущей демонстрации. Затем начальная нить присваивает только своему экземпляру 1. В первой демонстрационной области каждая сохраняет before и записывает свой номер.

Нить	Вход в первую область	После присваивания	Вход во вторую
0	1	0	0
1	0	1	1
2	0	2	2
3	0	3	3

Между областями последовательная часть видит только экземпляр начальной нити: threadValue = 0. Никакой автоматической суммы или сбора экземпляров других нитей нет. Во второй области каждый участник проверяет сохранённое значение против своего номера.

Условия сохранения нельзя опускать

Для схемы методички области не вложены в другую активную parallel-область, их команды имеют одинаковый размер, dynamic выключен. runLab3() сохраняет прежнюю настройку dynamic, отключает её и восстанавливает при выходе. Код также сравнивает фактические размеры двух команд. Между ними нет изменения режима или других параллельных конструкций.

В современных версиях стандарт перечисляет дополнительные условия, включая совпадение политики привязки и отсутствие некоторых промежуточных конструкций/операций, снимающих гарантию. Нельзя обещать переносимость сохранения при произвольной смене команд. Если нарушить условия, «но в прошлый раз сохранилось» не является достаточным обоснованием корректности.

6. copyin и copyprivate: разные рассылки

showCopyin(): на входе в parallel

До области начальная нить ставит threadValue = 1. copyin(threadValue) копирует это значение в экземпляры остальных участников. Все before должны быть равны 1. Затем каждая нить прибавляет свой номер: после получаются 1, 2, 3, 4. Последовательная часть видит экземпляр начальной нити, который остаётся равен 1.

```
threadValue = 1;
#pragma omp parallel num_threads(4) copyin(threadValue)
{
    threadValue += omp_get_thread_num();
}
```

copyin применяется к threadprivate, а не к произвольной обычной переменной. Он задаёт входное значение, не собирает результаты обратно и не объединяет экземпляры в одну общую переменную. Постоянство экземпляров задаёт threadprivate, а не сам copyin.

showCopyprivate(): после single внутри команды

В блоке parallel каждая нить создаёт обычные локальные value = -1 и source = -1. Исполнитель single устанавливает value = 42 и source в собственный номер. single copyprivate(value, source) рассылает обе переменные участникам. После этой конструкции все получают 42 и одинаковый номер источника.

```
int value = -1;
int source = -1;
#pragma omp single copyprivate(value, source)
{
    value = 42;
    source = omp_get_thread_num();
}
```

Источник выбирается реализацией single, не обязан быть нитью 0. У конструкции есть синхронизация завершения рассылки. copyprivate нельзя сочетать с nowait на той же single. Переменные-получатели должны быть приватными в охватывающем контексте или threadprivate согласно требованиям опции.

Вопрос	copyin	copyprivate
Когда	На входе в parallel	После выполнения single
От кого	От мастера команды	От выбранного исполнителя single
Куда	В экземпляры threadprivate	В допустимые приватные экземпляры участников

7. lastprivate и reduction

showLastprivate(): вернуть одно значение

value = -1 объявлена до цикла. Цикл от i = 0 до 11 вычисляет value = i * i в приватных экземплярах. lastprivate возвращает значение из последней **логической** итерации, то есть из i = 11: **121**. Неважно, какая нить последней закончила по настенным часам.

schedule(static, 1) при четырёх участниках раздаёт итерации по одной по кругу. Нить 0 получает 0, 4, 8; нить 1 - 1, 5, 9; нить 2 - 2, 6, 10; нить 3 - 3, 7, 11. Счётчик i приватен по правилам omp for. lastprivate не копирует исходное -1 на входе; перед чтением value каждой итерацией ей присваивается квадрат, поэтому неинициализированного чтения нет.

```
#pragma omp parallel for num_threads(4) \
    schedule(static, 1) lastprivate(value)
for (int i = 0; i < 12; i++)
{
    value = i * i;
}
```

showReduction(): объединить много результатов

sum = 0 имеет тип long long. reduction(+:sum) создаёт приватные аккумуляторы с нейтральным элементом 0; omp for делит диапазон 1...1000000. После цикла локальные суммы объединяются с исходной sum операцией сложения. Итог совпадает с N * (N + 1) / 2 = **500000500000**.

```
long long sum = 0;
#pragma omp parallel for num_threads(4) reduction(+:sum)
for (int i = 1; i <= 1000000; i++)
{
    sum += i;
}
```

В отличие от lastprivate, reduction использует результаты всех участников. Если sum до цикла была бы 10, начальное общее значение тоже участвовало бы в итоговом сложении. Приватные аккумуляторы + при этом всё равно начинаются с 0, а не каждый с 10. Для произведения нейтральный элемент - 1.

Для целых чисел этого примера переполнения long long нет. Для float/double порядок объединения может менять округление: (a + b) + c не всегда равно a + (b + c). Поэтому reduction не обещает побитовое совпадение вещественных результатов во всех запусках. int здесь недостаточен даже для правильного математического ответа.

8. Ответы на вопросы 1-3

1. Может ли одна переменная быть общей в одной части программы и локальной в другой?

Да, для обычной допустимой переменной. Атрибут совместного доступа относится к конкретной конструкции. В одной parallel-области можно указать shared(value), в другой private(value) или firstprivate(value). При приватизации имя внутри обозначает новые экземпляры; это не физическое превращение внешнего объекта в несколько частей.

Внешний объект продолжает существовать. Изменение копий private/firstprivate само по себе его не меняет. Существуют ограничения: например, threadprivate нельзя произвольно переобъявить shared или firstprivate в списке другой директивы. Для него предусмотрены свои правила и copyin.

2. Что произойдёт при одновременном обращении нескольких нитей к общей переменной?

Только чтение неизменяемого корректно инициализированного значения допустимо. Конфликтующие чтение/запись или запись/запись в одну память без необходимой синхронизации дают гонку. Результат нельзя считать корректно определённым. Нельзя говорить, что «обязательно будет последнее значение последней нити».

Пример counter++ может выполнить два чтения одного старого значения и две одинаковые записи: один инкремент потеряется. Для простого поддерживаемого обновления подходит atomic; для составного участка - critical или блокировка; для накопления - reduction. Барьер подходит для корректного разделения фаз записи и чтения, но не заменяет защиту конфликтующих записей в одной фазе.

3. Возможен ли конфликт при обращении к «одной и той же» локальной переменной?

Между независимыми приватными экземплярами - нет. Одинаковое имя в тексте программы не означает одинаковый адрес. Каждая нить может менять свою private-value без влияния на другие экземпляры.

Но если нескольким нитям передан адрес одного объекта, они обращаются уже к общей памяти по этому адресу. Приватные указатели тоже могут указывать на один common. Поэтому проверять нужно фактическую целевую память и упорядочивание доступа, а не только место объявления указателя.

Готовые короткие ответы

1: «Да, атрибут определяется конструкцией; private создаёт отдельные экземпляры». 2: «Общее чтение допустимо; конфликтующий доступ с записью требует синхронизации». 3: «Свои приватные объекты не конфликтуют, но указатели могут вести к общей памяти».

9. Ответы на вопросы 4-6

4. Как разослать всем нитям начальное значение на входе?

Для обычной переменной - **firstprivate**: создать приватные экземпляры с внешним начальным значением. Для уже объявленной **threadprivate** - **copyin**: скопировать экземпляр мастера в экземпляры остальных участников на входе в parallel. Если нужно только общее чтение, shared может быть достаточен, но это не создание независимых копий. Для значения, вычисленного внутри команды одним участником, подходит single copyprivate.

5. Можно ли сохранить локальные копии после parallel и как их использовать?

Обычные private/firstprivate не предоставляют автоматического сохранения и доступа к своим бывшим экземплярам. Для состояния по нитям используют threadprivate и соблюдают условия сохранения между областями: одинаковые команды, отключённый dynamic и остальные условия реализации/стандарта, разобранные выше.

Если нужен один результат последней логической итерации, выбирают lastprivate; для объединённого результата - reduction. Если нужны все значения независимо от повторного использования рабочих нитей, до выхода из области записывают их в отдельные элементы общего массива подходящего размера и времени жизни. Чтение итогового массива выполняют после завершения необходимых записей.

6. Чем copyin отличается от firstprivate?

Свойство	firstprivate	copyin
Переменная	Обычная, допустимая для конструкции	Только threadprivate
Действие	Создаёт приватные экземпляры и инициализирует их	Инициализирует экземпляры, заданные threadprivate
Источник значения	Внешний объект в контексте входящей нити	Экземпляр мастера новой команды
Время жизни	Связано с выполнением конструкции	Связано с экземпляром нити; сохранение имеет условия
Возврат всех результатов	Не делает	Тоже не делает

Практическая тонкость: нить 0 при firstprivate меняет новую копию обычной value и не меняет внешнюю. В threadprivate она меняет свой экземпляр, который видит и в последующей последовательной части. При этом copyin не является причиной сохранения: он только выполняет начальную передачу.

10. Типичные задачи и вопросы на защите

Нужно быстро предсказать вывод

value = 5, затем private(value), внутри value = 7. После области внешняя value остаётся 5. До присваивания 7 локальный int читать нельзя.

value = 5, затем firstprivate(value), внутри value++. Каждая копия становится 6, внешний объект остаётся 5. Если требуется вернуть одну копию, firstprivate сам по себе задачу не решает.

sum = 10, reduction(+:sum), каждая из четырёх нитей делает sum++. Локальные аккумуляторы начинаются с 0, каждый становится 1, итог равен 14. Это не 44 и не 4: исходное общее 10 участвует один раз.

Четыре нити проходят через critical и увеличивают счётчик. При правильном общем счётчике получим четыре увеличения, не одно. critical - по очереди всем; single - один выбранный участник один раз на команду.

После master поставить barrier: скопируется ли локальная value? Нет. Ожидание и видимость общей памяти не являются рассылкой приватных экземпляров. Для рассылки подходит single copyprivate.

Почему copyprivate source бывает не 0? single может исполнять любой участник. Единственное требование примера: после рассылки source у всех одинаков и входит в диапазон номеров команды.

Разбор служебных деталей программы

Анонимное пространство имён скрывает вспомогательные функции и threadValue от других исходных файлов. Именованный critical(outputLab3) защищает строки вывода, а reduction на errors защищает подсчёт нарушений. bool correct проверяет весь общий массив после области. Меню использует switch; cin.clear()/ignore() восстанавливают ввод, eof() позволяет закончить работу без бесконечного цикла.

Финальный чек-лист

Уметь назвать число экземпляров каждой переменной; различать входное и выходное копирование; объяснить 121 и 500000500000; объяснить условия threadprivate; найти гонку при замене reduction на shared; не читать private-int до присваивания; не путать приватный указатель с приватными данными; ответить на все шесть вопросов.

Debug и Release x64 проверены повторным запуском всех восьми примеров после доработки лабораторной №2. Все восемь встроенных проверок успешны. OpenMP и /utf-8 уже включены в vcxproj; на другом Windows-компьютере нужны установленный C++ toolchain, Windows SDK и поддерживающая библиотека runtime. Ничего из теоретических ответов не добавлено в консольную программу.